The `R-matrix-prop` Pipeline: A User's Guide

Adiabatic, SVD, and Hybrid Multichannel R-Matrix Propagation

August 19, 2026

Contents

1 Introduction and Big Picture

`R-matrix-prop` propagates a log-derivative/R-matrix outward in hyperradius R across a chain of "boxes" spanning $[x_{\text{Start}}, x_{\text{End}}]$, in a basis that is rebuilt (or interpolated) box by box, and matches the result to asymptotic reference functions at $R = x_{\text{End}}$ to extract $K/S/T$-matrices, phase shifts, and cross sections. It is built for multichannel few-body quantum scattering and bound-state problems in an *adiabatic hyperspherical representation*: a fast angular variable Ω (one or more hyperangles) is

diagonalized at fixed R to produce channel functions $\Phi_n(R;\Omega)$ and potential curves $U_n(R)$, and the slow radial variable R is then propagated through those channels.

Over its development this codebase accumulated *two largely independent design axes* that every new user needs to understand before touching anything:

1. **How is a box's basis represented radially?**
   - *Ordinary two-sided B-spline (FEM) box*, via `RMatPropCore.f90`'s `MakeBasis`/`CalcGamLam`. This is what `RMATPROP2016.f90` and `RMATPROPAdiabatic.f90` use.
   - *SVD/DVR box* ("slow variable discretization"), via `SVDChannelBasis.f90`/`GenericSVDChannelBasis.f90`'s `BuildSVDBox`/`BuildSVDBoxGeneric`. A box's radial dependence is represented on a handful of Gauss-Lobatto/Gauss-Radau DVR nodes instead of a dense B-spline mesh, exploiting a channel-overlap-matrix trick (Suno's method) so far fewer radial points are needed per box. This is what the `RMATPROPHybridSVD*` family uses.
2. **How is the adiabatic potential $U_n(R)/P_{mn}(R)/Q_{mn}(R)$ obtained at a given R?**
   - *Tabulate once, then interpolate* (`AdiabaticPotential.f90`): read a precomputed table (`Uad.dat`/`Pmat.dat`/`Qmat.dat`, generated once by **Adiabatic-Scattering-BoundStates**) and build a B-spline interpolant over it. Cheap per query, but only as accurate as the tabulation, and has a hard domain floor (Section 8).
   - *Evaluate directly at every quadrature point needed* (this session's `SechAdiabaticPotentialDirect.f90`, and **Adiabatic-Scattering-BoundStates**'s `DeltaScat.f90` for the delta-function benchmark): run the real per-R diagonalization (or, for `DeltaScat`, an exact closed-form formula) at the box's own Gauss-Legendre quadrature points, with no interpolation step and therefore no domain-floor problem at all. More expensive (one ARPACK diagonalization per point, $\sim$0.05 s each at reasonable angular resolution), but eliminates a whole category of near-origin bugs.

These two axes are *orthogonal*: any potential-evaluation strategy can in principle feed either box representation. In practice this codebase has historically paired "tabulate+B-spline box" (`RMATPROPAdiabatic.f90`) and "tabulate+SVD box" (`RMATPROPHybridSVD.f90`, now superseded), and this session added "direct+B-spline box" (`RMATPROPAdiabaticDirect3Boson.f90`) and already had "direct+SVD box" (`RMATPROPHybridSVDGeneric.f90`'s `RunHybridSVDGeneric`).

A third, more recent addition is a **pluggable physics interface** (`AdiabaticInterfaces.f90`/`AdiabaticSolverGeneric.f90`, in the shared `~/Documents/GitHub/lib/`) that lets a new physical problem be added by writing exactly three small subroutines (a potential evaluator, a grid maker, and a Robin-boundary-condition coefficient function) conforming to three abstract interfaces, instead of forking the entire adiabatic-solver machinery per problem the way **Adiabatic-Scattering-BoundStates**'s `adiabaticSolver1D.f90` historically had to be copy-pasted across $\sim$5 sibling repos. **This is the recommended entry point for solving a new Hamiltonian** – see Section 6.

2 Where Everything Lives

This pipeline spans three locations under `~/Documents/GitHub/`, which is itself *not* a single project but a workspace of ∼30 independent git repositories (see the top-level `CLAUDE.md` there for the full workspace map).

Location	Role	Key contents
R-matrix-prop	Main repo. The propagation engine, every driver program, and every physics-plugin file this guide describes (unless noted otherwise).	`RMatPropCore.f90`, `AdiabaticPotential.f90`, `SechFunctionPlugins.f90`, `SechAdiabaticPotentialDirect.f90`, `DeltaFunctionPlugins.f90`, `SVDChannelBasis.f90`, `GenericSVDChannelBasis.f90`, all `RMATPROP*.f90` drivers, `makefile`.
Adiabatic-Scattering-BoundStates (ASB)	Sibling repo. Generates the tabulated datasets `RMATPROPAdiabatic.x` consumes, and hosts the independent no-interpolation `DeltaScat` benchmark.	`adiabaticSolver1D.f90` (`OneDimChannels`, hardcoded potential + `GridMakerHHL`), `DriveAdiabaticSolver1D.f90` (driver, writes `Uad.dat` etc.), `NewFitProg.f` (long-range fit → `Fit.data`/`FitLeff.data`), `DeltaScat/` (self-contained no-interpolation benchmark, own copy of `RMatPropCore.f90`).
`~/Documents/GitHub/lib/`	Canonical shared numerics, used by many unrelated repos in the workspace – migrate stale local copies here, don't fork.	`Bsplines.f90`, `Quadrature.f90`, `matrix_stuff.f90`, `besselnew.f90`, `milne_core.f90` (has a generic `GridMaker`), `AdiabaticInterfaces.f90`, `AdiabaticSolverGeneric.f90`.
`~/Documents/GitHub/interpolation/`	Canonical B-spline interpolation library (module `InterpType`), used by `AdiabaticPotential.f90`.	`Interpolation.f90`.

Important: R-matrix-prop keeps its own *local copy* of ASB's `adiabaticSolver1D.f90` (needed because `AdiabaticSolverGeneric.f90`'s `OneDimChannelsGeneric` links directly against several bare subroutines defined inside it – `CalcOverlap`, `CalcPHF`, `FixPhase`, `CalcEigenErrors` – and a separate local copy avoided a link-time symbol collision when this was first set up). **The two copies must be kept in sync by hand.** If you fix a bug in one (as this session did for `GridMakerHHL`'s domain-edge case, Section 8), port the fix to the other.

3 Building the Codebase

3.1 The `makefile` pattern

Every executable follows the same three-part pattern:

```
FOO_OBJS = besselnew.o Bsplines.o matrix_stuff.o Quadrature.o Interpolation.o \
         RMatPropCore.o <physics-specific>.o Foo.o

Foo.x: ${FOO_OBJS}
```

```
	${CMP} ${DEBUG} ${FOO_OBJS} ${INCLUDE} ${ARPACK} ${LAPACK} ${CMPFLAGS} ${FORCEDP} -o Foo.x

Foo.o: Foo.f90 RMatPropCore.o <physics-specific>.o
	${CMP} ${DEBUG} ${FORCEDP} ${CMPFLAGS} -c Foo.f90
```

To add a new driver, copy an existing `*_OBJS/.x/.o` block for the closest existing driver and edit the object list and source filename.

`makefile` and `Makefile` are the *same file* (case-insensitive filesystem, same inode) – don't edit one expecting the other to diverge.

3.2 Compiler flags

`DEBUG = -fcheck=all` **unconditionally**, on every target. This is deliberate: there is no automated test suite for this codebase, so bounds/shape runtime checking is the main safety net. Don't strip it for a "performance run" without checking whether the user actually wants that trade-off. On macOS, LAPACK/BLAS come from `-framework Accelerate`; ARPACK (needed by every adiabatic-diagonalization path) from Homebrew, `-L/opt/homebrew/lib/ -larpack -I/opt/homebrew/include`.

3.3 Running

Almost every driver reads its parameters from a hardcoded input filename via plain `OPEN(unit=7,file='Foo.inp',status='old')` – **not** from stdin redirection or a command-line argument, despite what `./Foo.x < Foo.inp` might suggest. Several drivers (`RMATPROPAdiabatic.x`, `DeltaScat.x`) share one hardcoded input filename across multiple physical configurations (e.g. `RMATPROPAdiabatic.inp`), so switching datasets means *copying* the config file you want over that hardcoded name, running, and copying it back – there is no built-in multi-config support. `ASB`'s `DriveAdiabaticSolver1D.f90` and `RMATPROPHybridSVDGeneric*` family are the exceptions: they genuinely read from stdin or their own uniquely-named `.inp` file.

3.4 Build gotchas

- `GridMaker` **name collisions.** At least three incompatible subroutines named `GridMaker` exist in files that can end up linked into the same executable: `GridMaker(grid,numpts,E1,E2,scale)` (the generic multi-mode one, in `lib/milne_core.f90` and duplicated locally in `RMatPropCore.f90`), and `GridMaker(mu,R,r0,xNumPoints,xMin,xMax,xPoints,CalcNewBasisFunc)` (the hyperspherical angular-grid one, in `adiabaticSolver1D.f90`). Fortran has no namespacing for bare external subroutines, so linking both into one executable is a hard duplicate-symbol error. This session hit exactly this when adding a generic-grid `GridMaker` call to ASB's `DriveAdiabaticSolver1D.f90` (which already links `adiabaticSolver1D.f90`); the fix was to rename the newly-added copy to `GridMakerScale` (see `DriveAdiabaticSolver1D.f90`'s own header comment). **Before adding any new source file to a build target, grep the whole object list for the subroutine names you're about to introduce.**
- `GaussLobatto.dat`/**Gauss-Radau tables.** `GetGaussLobattoFactors` (used by the SVD/DVR box builders) looks up nodes/weights by exact point count in a text table, `GaussLobatto.dat` – it does *not* compute them on the fly, and only a handful of point

counts are pre-tabulated (6, 8, 10, 12, ..., 200). The `'LobattoDirichlet' GridType` (Section 5) needs *both* L and $L+1$ present simultaneously, which *no pair* in the shipped table satisfies – this session had to compute and append two new blocks (N=20,21) with a short Python/`numpy.polynomial` script, matching the file's existing tab-separated format exactly. If you pick a new L, check the table first. Gauss-Radau nodes, by contrast, are computed on the fly (via `GetGaussRadauFactors` in `SVDChannelBasis.f90`, a DSTEV-based routine added in this project's own history) – no table dependency there.

- `Legendre.dat` must exist in the working directory at runtime (read by `GetGaussFactors`); it's not regenerated.

4 File and Subroutine Reference

This section is organized bottom-up: shared numerics, then `R-matrix-prop`'s own physics-independent engine, then the physics-provider layer (where a new problem's code lives), then the driver programs, then ASB's dataset-generation side.

4.1 Layer 0 – Shared numerics (`~/Documents/GitHub/lib/`)

`Bsplines.f90`

`CalcBasisFuncsBP(Left,Right,kLeft,kRight,Order,xPoints,LegPoints,xLeg,MatrixDim, xBounds,xNumPoints,Deriv,u)`: builds B-spline basis function values (`Deriv=0`) or second derivatives (`Deriv=2`) at every Gauss-Legendre quadrature point, respecting the boundary condition codes:

Code	Meaning
0	Dirichlet ($u=0$ at that edge) – single boundary function, built from B-spline index 2 only
1	Neumann ($u'=0$) – single boundary function, B_1+B_2
2	Open – *both* B_1, B_2 independently retained (no constraint imposed at all)
3	Robin ($u'=k\,u$) – single boundary function, $B_1+B_2/\text{const}(k)$

Codes 0 and 3 give the *same* knot-vector size (one fewer function than code 2); the difference is purely which linear combination of B_1, B_2 is used, i.e. what value that one remaining function actually takes at the edge. Code 2's "do nothing, keep both" is what `R-matrix-prop` actually uses for box 1's inner edge in modern drivers – the boundary condition is imposed *variationally* afterward by `PartitionAndEliminate`, not by the basis construction itself (Section 5).

`Quadrature.f90`

`GetGaussFactors(File,Points,x,w)` – Gauss-Legendre nodes/weights, computed on the fly.
`GetGaussLobattoFactors(File,Points,x,w)` – table lookup in `GaussLobatto.dat` (Section 3's gotcha above).
Also contains Cash-Karp ODE coefficients (`initCashKarp`) – unused by anything in this pipeline; only relevant to Milne-equation solvers elsewhere in the workspace.

`matrix_stuff.f90`

`MyDsband` – wraps ARPACK's `dsband` (banded shift-invert Lanczos eigensolver); this is what every adiabatic per-R diagonalization ultimately calls.
`CalcEigenErrors`, `FixPhase` (enforces sign continuity of eigenvectors across successive R –

critical: DSYGV/ARPACK's per-R eigenvector sign is arbitrary, and an uncorrected sign flip between adjacent R corrupts any cross-R overlap/coupling calculation), `CalcOverlap`, `CalcPHF` (Hellmann-Feynman P-matrix, $P_{mn} = \langle \Phi_m | \partial_R \Phi_n \rangle$).

`milne_core.f90`

`GridMaker(grid,numpts,E1,E2,scale)`: the canonical general-purpose 1D grid generator, 8 modes selected by the `scale` string:

`scale`	Formula ($t = (i-1)/(\text{numpts}-1)$)
`"linear"`	$E_1 + t\,(E_2 - E_1)$
`"log"`	$\exp(\ln E_1 + t(\ln E_2 - \ln E_1))$ (requires $E_1, E_2 > 0$)
`"neglog"`	log-spacing for negative E_1, E_2
`"quadratic"`	$E_1 + t^2(E_2 - E_1)$
`"cubic"`, `"quartic"`	$E_1 + t^3(E_2 - E_1)$, $E_1 + t^4(E_2 - E_1)$
`"sqrroot"`, `"cuberoot"`	$E_1 + t^{1/2}(E_2 - E_1)$, $E_1 + t^{1/3}(E_2 - E_1)$

Beware: an unrecognized `scale` string silently leaves the array uninitialized past index 1 – there is no `ELSE`/error branch. Also beware the *unrelated*, differently-parameterized `GridMakerQuadratic(xNumPoints,xMin,xMax,xPoints)` found in some other workspace repos (`LiScattering/CalcBound1D.f90`, `TuneVeff/CalcBound1D.f90`): that one's formula is $t^2\, x_{\max} + x_{\min}$, not $x_{\min} + t^2(x_{\max} - x_{\min})$ – they only agree when $x_{\min} = 0$. Don't mix them up.

`AdiabaticInterfaces.f90`

Three `ABSTRACT INTERFACE` declarations that any new-physics plugin must conform to:

```
SUBROUTINE PotentialProc(R,LegPoints,xNumPoints,x0,Pot,dPot)
  ! Pot/dPot(LegPoints,xNumPoints) = V and dV/dR at the already-mapped
  ! angular quadrature points x0(lx,kx), at fixed hyperradius R.

SUBROUTINE GridMakerProcI(R,xNumPoints,xMin,xMax,xPoints,CalcNewBasisFunc)
  ! Returns the angular knot grid xPoints at radius R (may depend on R).

SUBROUTINE RobinCoeffProc(R,kLeft,kRight)
  ! Returns the Robin (Left/Right=3) log-derivative coefficients at R.
  ! For non-Robin BC codes (0/1/2), return the sentinel 1d20 -- never read.
```

`AdiabaticSolverGeneric.f90`

`OneDimChannelsGeneric(NumStates,PsiDim,PsiFlag,CouplingFlag,LegendreFile,LegPoints,Shift,Order,Left,Right,mu,xNumPoints,xMin,xMax,R,RSteps,GridRebuildEveryR,RAnchor,MyPotential,MyGridMaker,MyRobinCoeff,Ufile,Pfile,Qfile,dPfile,VQfile,Uad,Psi,S_out,datadir)` – a generalized, pluggable drop-in analog of ASB's `OneDimChannels`: same ARPACK shift-invert diagonalization, `FixPhase`, `CalcPHF` machinery, but the potential/grid/BC come from the three caller-supplied procedures above instead of a hardcoded `use potential` + `GridMakerHHL` pipeline. `GridRebuildEveryR` **is the single most important argument to get right** – see Section 8.

`CalcHamiltonianGeneric` – the potential-agnostic Hamiltonian-matrix assembly `OneDimChannelsGeneric` calls internally; structurally identical to ASB's own `CalcHamiltonian` but takes `Pot`/`dPot` values from `MyPotential` instead of a hardcoded pairwise-distance calculation.

4.2 Layer 1 – R-matrix-prop's core engine (RMatPropCore.f90)

Everything in this file is **physics-independent**: it operates purely on BPD%Pot/BPD%Pcoup (already filled by whichever physics-provider routine you call) and never evaluates a potential itself.

Module GlobalVars Run-wide scalars: NumChannels, xStart, xEnd, reducedmass, AlphaFactor, EffDim, NumBoxes, Energy, Pi. AlphaFactor **convention**, verbatim from this module's own comment: "This calculation ALWAYS sets AlphaFactor = (EffDim-1)/2 and StartBC = 0" – see Section 5 for what this actually means and when you'd deviate from it. (LegPoints/xLeg/wLeg/LegendreFile live in the *separate* Quadrature module, not here – an easy mix-up.)

Module DataStructures The core derived types:

```
TYPE BPData ! one box's basis + potential
  INTEGER xNumPoints, xDim, Left, Right, Order, NumChannels, MatrixDim
  DOUBLE PRECISION, ALLOCATABLE :: u(:,:,:), ux(:,:,:), xPoints(:), x(:,:)
  DOUBLE PRECISION, ALLOCATABLE :: Pot(:,:,:,:), Pcoup(:,:,:,:), lam(:)
  DOUBLE PRECISION :: kl, kr, xl, xr
END TYPE BPData

TYPE GenEigVal ! one box's assembled generalized-eigenvalue pieces
  DOUBLE PRECISION, ALLOCATABLE :: Gam(:,:), Gam0(:,:), Overlap(:,:), Lam(:,:)
  ! Gam0/Overlap: energy-independent, built once and cached; Gam = Gam0 + Energy*Overlap
  ! (via CombineGam) is what actually gets fed to PartitionAndEliminate.
END TYPE GenEigVal

TYPE BoxData ! one box's propagated log-derivative amplitudes
  INTEGER :: NumOpenL, NumOpenR, betaMax
  DOUBLE PRECISION, ALLOCATABLE :: Z(:,:), ZP(:,:), ...
END TYPE BoxData

TYPE ScatData ! final K/R/S/T matrices
  DOUBLE PRECISION, ALLOCATABLE :: K(:,:), R(:,:), f(:,:), sigma(:,:)
  COMPLEX*8, ALLOCATABLE :: S(:,:), T(:,:)
END TYPE ScatData
```

Plus Allocate*/DeAllocate* for each.

MakeBasis(BPD) Builds an ordinary two-sided B-spline basis (via CalcBasisFuncsBP) and fills BPD%u/BPD%ux/BPD%x for one box, given BPD%xPoints (the box's internal knot grid) already set.

CalcGamLam(BPD,EIG,NumOpenL,NumOpenR) Assembles EIG%Gam0/Overlap/Lam from BPD%Pot/Pcoup (which some physics-provider routine – SetAdiabaticPotential, SetAdiabaticPotentialDirect, etc. – must already have filled). Physics-independent; the same routine serves every driver in the B-spline-box family.

CombineGam(EIG,Energy) Gam = Gam0 + Energy × Overlap.

PartitionAndEliminate(BPD,EIG,NumOpenL,NumOpenR,evalRed,evecRed,LamooDiag) / PartitionAndElimin
Variational elimination of closed/interior degrees of freedom, reducing a box to its retained boundary amplitudes. NumOpenL=0 is how a box's *left* boundary condition is imposed in the modern (Left=2) box-1 convention – eliminating a DOF with no coupling out that side is the R-matrix-propagation equivalent of applying whatever boundary condition the underlying

basis naturally encodes there. The `Full` variant also returns eigenvector data needed to reconstruct actual spline coefficients (used only by the `PlotWavefunction*` diagnostics).

`BoxMatch(BA,BB,BPD,evalRed,evecRed,LamooDiag,dim,alphafact)` **(via** `BoxMatchEigensystem`**)** Propagates the R-matrix/log-derivative from box `BA` to box `BB`.

`CalcK(B,BPD,SD,mu,d,alpha,EE,Eth)` Final asymptotic matching at $R = x_{\text{End}}$: projects the propagated log-derivative onto hyperspherical-Bessel/Riccati-Bessel reference functions (via `besselnew.f90`'s `hyperrjry`) to produce the K-matrix, using each channel's threshold energy `Eth` and effective angular momentum (`BPD%lam`, usually called `Leff` by the caller). **This is the piece that changes if your new problem's long-range asymptotics aren't of this hyperspherical-Bessel/Coulomb form** – see Section 6.

`BuildBox1CombinedBasis/CalcGamLamBox1/CoulombRegularNumeric/ BuildBox1NeumannBasis` **Legacy.** An older, energy-dependent box-1 mechanism that builds a Robin-type combined basis function (via a closed-form `hyperrjry`-based log-derivative, or numerical RK4 integration of the true near-origin potential for a genuine Coulomb-divergent channel) to approximate the box-1 regularity condition at finite `xl`. `RMATPROPAdiabatic.f90` **no longer calls any of this** – its own header comment states it is "confirmed equivalent to . . . a plain Neumann box-1 basis" and now uses the simpler `Left=2` + `PartitionAndEliminate` approach unconditionally (Section 5). This machinery is kept for reference and for any driver that still needs a literal Robin condition at a genuinely nonzero `xl`.

`GridMaker(grid,numpts,E1,E2,scale)`, `GridMakerLinear(xNumPoints,x1,x2,xPoints)` Local copies of the shared utilities described above (see the name-collision warning in Section 3).

4.3 Layer 2 – Physics providers (fill `BPD%Pot`/`BPD%Pcoup`, or build an SVD box directly)

This is the layer you replace or extend for a new Hamiltonian.

`AdiabaticPotential.f90` – **tabulate + interpolate**

`ReadAdiabaticData(DataDir,NumChannels,NumDataPoints,muLocal,alpha,ddim,Threshold,Leff, UInterp,PInterp,QInterp)`: reads `DataDir/{Fit.data,masses.dat,Uad.dat,Pmat.dat, Qmat.dat,FitLeff.data}` and builds B-spline interpolants (order-2, piecewise linear, to avoid Runge ringing) via `~/Documents/GitHub/interpolation`'s `InterpType` module.
`SetAdiabaticPotential(BPD,muLocal,UInterp,PInterp,QInterp,CoulombC)`: evaluates those interpolants at `BPD`'s own Gauss-Legendre quadrature points, filling `Pot`$(m,n) = \tilde{Q}_{mn}(R)/2\mu + \delta_{mn}U_m(R)$ and `Pcoup` $= P(R)$. `CoulombC` (per-channel, usually all zero) switches individual channels to an analytic near-origin series (`AnalyticComboNearOrigin`) below `RcutoffAnalytic=0.01`, for genuinely Coulomb-divergent channels only – **do not** infer this from `Threshold` < 0 alone; a short-range-potential bound-pair channel also has negative threshold without being Coulomb-divergent (Section 8).

`SechFunctionPlugins.f90` – **example** `AdiabaticInterfaces` **plugin**

`SechPotential`, `SechGridMaker` (an exact replica of ASB's `GridMakerHHL`), `SechRobinCoeff` (trivial sentinel, non-Robin BC). Physics: equal-mass 3-body sech^2-well problem, pairwise distances $r_{ij} = \sqrt{2/\sqrt{3}}\,R\,|\cos(\phi - \text{offset}_{ij})|$. Read this file as the **template** for a new plugin.

`SechAdiabaticPotentialDirect.f90` – **no-interpolation adiabatic potential for B-spline boxes**

`SetAdiabaticPotentialDirect(BPD,muLocal)`: builds the box's own quadrature-point R array (matching `BPD%x(lx,kx)`'s layout exactly), calls `OneDimChannelsGeneric` once for the *whole box* (`CouplingFlag=1` for Hellmann-Feynman P/Q, `GridRebuildEveryR=.TRUE.`) with `SechFunctionPlugins`, writes P/Q to scratch files, and re-reads them with *exactly* `AdiabaticPotential.f90`'s own reading convention so the sign/normalization matches automatically. Direct, drop-in replacement for `SetAdiabaticPotential` in any driver, at the cost of $\sim$0.05 s per quadrature point instead of an interpolation lookup – **never call this once per energy**; cache its output per box exactly once (Section 7).

`DeltaFunctionPlugins.f90` Closed-form plugin for the equal-mass three-boson delta-function benchmark: `DeltaZeroPotential` ($V \equiv 0$ identically), `DeltaWedgeGridMaker` (fixed uniform grid over $\phi \in [0, \pi/6]$), `DeltaRobinCoeff` ($k_{\text{right}} = \sqrt{2\mu}\, R$, encoding the contact interaction as a genuine Robin BC).

`SVDChannelBasis.f90` / `GenericSVDChannelBasis.f90`
`BuildSVDBox/BuildSVDBoxGeneric(a1,a2,L,NumStates,...,GridType,BPD,EIG,...)`: construct one SVD/DVR box's `Gam0/Overlap/Lam` *directly* (not via `BPD%Pot/Pcoup` – a structurally different mechanism from the B-spline path), diagonalizing at Gauss-Lobatto or Gauss-Radau DVR nodes and using a channel-overlap-matrix trick to avoid needing a separate radial basis. `GridType` $\in$ {`'Radau'`, `'Lobatto'`, `'LobattoDirichlet'`} – see Section 5. The `Generic` variant takes `MyPotential/MyGridMaker/MyRobinCoeff` (i.e. is pluggable, calling `OneDimChannelsGeneric` internally); the plain version is hardcoded to ASB's `OneDimChannels`.

4.4 Layer 3 – Driver programs

Driver	Box type	Potential source	Notes
`RMATPROP2016.x`	B-spline	Synthetic Baluja multipole potential (`Baluja` module)	Reproduces the Baluja et al. CPC (1982) benchmark. Reads its own `RMATPROP.inp`-style format via `ReadGlobal`.
`RMATPROPAdiabatic.x`	B-spline	Tabulated + interpolated (`AdiabaticPotential.f90`)	Box 1: `Left=2`, standard elimination (not the legacy Robin mechanism). Last box rebuilt fresh every energy (cheap here, since interpolation lookup is cheap). Energy-independent boxes' `Gam0/Overlap/Lam` cached once.

Driver	Box type	Potential source	Notes
`RMATPROPAdiabaticDirect3Boson.x`	B-spline	Direct (`SechAdiabaticPotentialDirect.f90`)	**This session.** Same structure as `RMATPROPAdiabatic.x` except the last box is *also* cached (via a dedicated `BPDLast`, built once) – essential here since re-evaluating the potential is $\sim 1000\times$ more expensive than an interpolation lookup. Hardcoded to the equal-mass 3-identical- boson sech^2 problem, domain $[0, \pi/6]$, `Left=Right=1`. Use this file as the **template** for a new no-interpolation B-spline-box driver.
`RMATPROPHybridSVD.x`	SVD (+ B-spline tail)	Tabulated, via `SVDChannelBasis.f90`'s own hard-coded `OneDimChannels` call	**Superseded** by the generic pipeline below; kept buildable but not the recommended starting point.
`RMATPROPHybridSVDGeneric.f90` (module `RMATPROPHybridSVDGenericMod`, subroutine `RunHybridSVDGeneric`)	All-SVD	Pluggable (`MyPotential/MyGridMaker/ MyRobinCoeff`)	**Not a `PROGRAM`** – a reusable subroutine (Fortran `PROGRAM`s can't take procedure arguments), called by a thin per-physics driver. Takes `Box1GridType` (`'Radau'` or `'LobattoDirichlet'`, added this session – see Section 5). Already wires up the general multichannel unitarity check ($\|S^\dagger S - I\|_\infty$), written to `Recombination.dat`.
`RMATPROPHybridSVDGenericDelta.f90`	All-SVD	`DeltaFunctionPlugins.f90`	Thin driver instantiating `RunHybridSVDGeneric` for the delta-function benchmark, `Box1GridType= 'Radau'`. **Validated**: reproduces `RMATPROPHybridSVDRobin.x`'s own results exactly.
`RMATPROPHybridSVDGenericSech.f90`	All-SVD	`SechFunctionPlugins.f90`	Thin driver for the (2-fermion+1-distinguishable) sech^2 case, domain $[0, \pi/2]$.

Driver	Box type	Potential source	Notes
`RMATPROPHybridSVDGenericSech3Boson.f90`	All-SVD	`SechFunctionPlugins.f90`	Thin driver for the equal-mass *3-identical-boson* sech^2 case, domain $[0, \pi/6]$, `Left1D=Right1D=1` (Neumann-Neumann, even-parity BC at both edges – `SechGridMaker`'s own `xMax` $\leq \phi_{23}$ branch already handles the domain's right edge coinciding with the coalescence point). `NumChannels/mu/ Threshold/Leff` are read straight from `DataDir`'s own `Fit.data/ FitLeff.data`, so the same driver runs any `3bodydata_sech2_3boson*` dataset – see the naming convention note below. Independent cross-check of `RMATPROPAdiabatic.x` on the identical physics (Section 6.1).
`SVDBound.x`	Single DVR grid	Tabulated	Bound-state solver, one direct diagonalization, no box-chaining.
`SVDRmat.x`	Single DVR grid	Tabulated	Wigner-Eisenbud-form R-matrix (pole sum), not the log-derivative propagation approach used elsewhere.
`BSplineFree.x`	B-spline	$V = 0$	Free-particle sanity check against the exact solution; deliberately avoids the modern box-1 machinery (2019-era recipe).
`PlotWavefunction*.x`	(diagnostic)	n/a	Standalone single-energy/single-box wavefunction reconstructions, validated against independent RK45 integration.
`PlotWavefunctionBoxByBox.f90`	(diagnostic)	n/a	Multi-box wavefunction reconstruction, adiabatic B-spline chain and SVD chain side by side, using *raw* `evecRed/ evalRed` (not `BoxMatch`'s own unit-normalized `Zf/Zfp`) with a value-*and*-derivative box-interface match, correctly flipping the outward-normal sign of the log-derivative at a box's *left* edge only (Section 6.1).
`CompareRmatrixSVDvsAdiabatic.f90` / `CompareRmatrixMultichannel.f90`	(diagnostic)	both paths, single box	A/B comparators isolating box *construction* from `BoxMatch` chaining and `CalcK`'s asymptotic matching: the former diffs the adiabatic-tabulated vs. SVD-direct R-matrix/wavefunction at one box; the latter tests whether embedding a channel in a larger `NumStates`-channel SVD construction changes its own R-matrix relative to building it alone.

Driver	Box type	Potential source	Notes
`TestBoxCountRmat.f90`	(diagnostic)	SVD	Box-count-independence sanity check (1/2/3 boxes spanning the same total range must agree to machine precision) – isolates `BoxMatch` chaining correctness from the separate wavefunction-reconstruction-only issue above.

Dataset naming convention (`3bodydata_sech2_3boson*`): a trailing `_{10,20}ch` means `NumChannels` retained in that dataset's own long-range fit (default, unsuffixed, is 5); a trailing `_rmax200` means `NewFitProg.f`'s long-range fit window was `Rdelay` units below R=200 (i.e. $R \in [191, 201]$) instead of the default $R \in [40, 50]$. Both axes matter independently for `FitLeff.data`'s `Leff` (see Section 6.1) – when comparing `RMATPROPAdiabatic.x` against `RMATPROPHybridSVDGenericSech3Boson.x` on the same physics, point both drivers at datasets matching on *both* axes, or an apparent disagreement may just be inconsistent auxiliary data.

4.5 Layer 4 – ASB: dataset generation for the tabulate+interpolate path

Only relevant if you're using `AdiabaticPotential.f90` (i.e. `RMATPROPAdiabatic.x` or a sibling); skip this if you're going the "direct" or SVD-pluggable route.

`adiabaticSolver1D.f90 OneDimChannels` (the hardcoded analog of `OneDimChannelsGeneric`: same ARPACK/`FixPhase`/`CalcPHF` machinery, but potential via `use potential+sumpairwisepot` and grid via the hardcoded `GridMakerHHL`). `CalcHamiltonian`, `GridMakerHHL` (non-uniform ϕ grid, refines near coalescence points, narrows with R as $\delta x = r_0/R$). **See Section 8 for the domain-edge bug fixed here this session.**

`DriveAdiabaticSolver1D.f90 PROGRAM`, reads its parameter file from stdin, writes `3bodydata/{Uad,Pmat,Qmat,dPmat,VQmat,masses}.dat`. The R-tabulation grid is now built via a locally-added `GridMakerScale(R,RSteps,RMin,RMax,"quadratic")` (session fix – was uniform, which left almost no resolution near $R = 0$; see Section 8).

`NewFitProg.f` Fits the long-range tail of `Uad.dat`/`Pmat.dat`/ `VQmat.dat` (over the last `Rdelay=10` units of R) to produce `Fit.data` (`NumChannels`, `NumDataPoints`, `alpha=(EffDim-1)/2`, `ddim=EffDim=2` – hardcoded, this repo's fixed convention) and `FitLeff.data` (per-channel `Threshold`, `Leff`, used by `CalcK`'s asymptotic matching).

`DeltaScat/` Self-contained, **independently validated** no-interpolation reference: evaluates the delta-function potential in closed form (`KMSFormulas.f/AnalyticDeltaPotential.f90`) directly at quadrature points, using its *own* local copy of `RMatPropCore.f90`. Box 1: `Left=Right=2`, `xStart=0.0` exactly – proof that this mechanism is sound at the literal origin. Validated against the exact closed-form Mehta-Shepard atom-dimer K-matrix (scattering length to ∼0.5%) and McGuire's exact-elastic-unitarity result ($1 - |S_{11}|^2 \to 0$, confirmed to 10^{-10}–10^{-6} across the full multichannel positive-energy range). **Read this file first** if you want to see the "ideal" (no tabulation, no interpolation) box-construction pattern before building your own. Its own input file, `DeltaScat.inp`, is hardcoded-by-name like `RMATPROPAdiabatic.inp` – swap configs by copying over it.

5 Box 1: The Regularity/Boundary Condition at $R = x_{\text{Start}}$

This is the single most confusing piece of the whole pipeline, and the source of most of this session's debugging. Two *independent* choices interact here and are easy to conflate:

Choice 1: `AlphaFactor` – which radial wavefunction are you solving for? $\Psi(R,\Omega) = \sum_n R^{-\alpha} F_n(R)\Phi_n(R,\Omega)$. Two conventional choices:

- $\alpha = 0$: F_n *is* the true wavefunction. The kinetic operator keeps a first- derivative term. Regularity at $R \to 0$ is an *open* condition – the true wavefunction just has to stay finite, with no value constraint.
- $\alpha = (\text{EffDim} - 1)/2$: F_n is the *reduced* wavefunction $u(R) \sim R^{\frac{\text{EffDim}-1}{2}}\Psi(R)$. This eliminates the first-derivative term from the kinetic operator (manifestly symmetric Hamiltonian, see `AdiabaticHypersphericalNotes.md` in **Adiabatic-Scattering-BoundStates** for the full derivation), at the cost of introducing a genuine *Dirichlet* regularity condition, $u(0) = 0$, since $u \sim R^{\alpha+L}$ for a channel of effective angular momentum $L \geq 0$.

`R-matrix-prop` **always uses the second convention** (`AlphaFactor` $= (\text{EffDim} - 1)/2$, `StartBC`$=0$) – see `GlobalVars`'s own comment. The delta-function benchmark's *SVD*-side plugins (`RMATPROPHybridSVDGenericDelta.f90`) are the one documented exception, using $\alpha = 0$ deliberately for that specific reduced-coordinate problem.

Choice 2: how is that boundary condition actually imposed, mechanically? Three mechanisms exist in this codebase, and **only one of them can literally represent** $x_{\text{Start}} = 0$:

Mechanism	Correct for	How
B-spline `Left=2` + `PartitionAndEliminate`	Either α convention	Both boundary functions retained (no constraint baked into the basis); the boundary DOF is then *variationally eliminated* (`NumOpenL=0`). Works at literal $R = 0$ because the box's Gauss-Legendre quadrature points are always strictly interior to $[x_l, x_r]$ – $R = 0$ is never actually evaluated. **This is what `RMATPROPAdiabatic.f90` and `DeltaScat.f90` both actually use**, confirmed identical box geometry/grid construction between them this session.
`GridType='Radau'` (SVD/DVR)	$\alpha = 0$ (open/regularity)	Gauss-Radau quadrature fixes only the *right* endpoint; a_1 is structurally never a node at all, so the retained basis is open/unconstrained there – correct only when that *is* the physical condition.
`GridType='LobattoDirichlet'` (SVD/DVR)	$\alpha = (\text{EffDim} - 1)/2$ (Dirichlet)	a_1 is a *formal* Lobatto node (needed to build a correct Lagrange derivative operator) whose DOF is then eliminated – true $u(a_1) = 0$. Like the B-spline case, `OneDimChannels/ OneDimChannelsGeneric` is *never actually evaluated* at a_1 itself, regardless of which of these two `GridTypes` is used.
`BuildBox1CombinedBasis` (`Left=3`, Robin, legacy)	Neither, in general	Computes a *finite*-radius log-derivative k_{left} via `hyperrjry` (closed-form) that *approximates* the $\alpha = (\text{EffDim} - 1)/2$ Dirichlet limit as $x_l \to 0$ ($k_{\text{left}} = L_{\text{eff}}/x_l \to \infty$). **Cannot represent $x_l = 0$ at all** (literal $L/0$), and for large L_{eff} is numerically fragile even at small nonzero x_l (special-function underflow at small argument) – this session hit exactly this failure mode (astronomic, nonsensical K-matrix values) before realizing `RMATPROPAdiabatic.f90` doesn't actually call this path any more. **Not needed for new work**; understand it only if you're reading old code or reviving the Robin-matched approach for a case that genuinely needs a nonzero-radius approximate boundary (e.g. a hard inner wall away from the origin).

Radau cannot represent Dirichlet, structurally – Gauss-Radau quadrature has no node at a_1 at all, so there is no DOF there to eliminate; the two `GridTypes` are not interchangeable substitutes for each other despite superficially both being "avoid evaluating at a_1" tricks. Pick the one matching

your `AlphaFactor` convention.

The domain edge doesn't have to be $R = 0$. Everything above generalizes to *any* box-1 inner edge, not just the true origin – e.g. a genuinely nonzero inner cutoff imposed for physical reasons. What's specific to $R = 0$ is only the additional hard numerical constraint that the hyperspherical kinetic prefactor $m = -1/(2\mu R^2)$ (Section 8) makes literal $R = 0$ unusable for *any* mechanism that evaluates the potential exactly there.

6 How to Solve a New Hamiltonian

This section walks through the same steps this session actually followed building the sech^2 3-identical-boson case, generalized. Read Sections 5 and 8 first – most of the mistakes described there are exactly the ones a new problem will re-expose.

Step 1: Coordinates

Work out your mass-scaled Jacobi transformation and the mapping from your chosen hyperangle(s) Ω to the physical pairwise/interparticle distances the potential actually depends on (e.g. $r_{ij}(R, \phi)$ for a 3-body 1D problem). `AdiabaticHypersphericalNotes.md` (in **Adiabatic-Scattering-BoundStates**) derives the general d-dimensional Laplacian transformation and the Hamiltonian-matrix-element expressions this whole pipeline implements; read it once before writing a new plugin, it will save you from re-deriving the $\tilde{Q} = P^2 + \partial_R P$ decomposition and the $\alpha = (\text{EffDim} - 1)/2$ manifestly-symmetric form yourself.

Step 2: Domain and boundary conditions

- Decide whether you need the *full* angular domain or can exploit permutation/parity symmetry to restrict to a smaller fundamental domain (as this session did, reducing the 3-identical-boson sech^2 problem from $[0, \pi/2]$ to $[0, \pi/6]$, mirroring what the delta-function benchmark already does for the same symmetry reason). A domain-edge coalescence point requires a grid maker that refines toward that edge, not around an interior point – `SechFunctionPlugins.f90`'s `SechGridMaker` vs. `SechWedgeGridMaker` (the latter is the edge-refining variant this session started building for the reduced domain) are templates for the two cases respectively. **If you reuse ASB's own `GridMakerHHL` with a domain edge coinciding exactly with its hardcoded interior coalescence angle $\phi_{23} = \pi/6$, you will hit the degenerate-segment bug this session found and fixed** (Section 8) – either use the fixed version (both repo copies were patched) or write your own domain-edge-aware grid maker.
- Work out what boundary condition your problem's symmetry (particle exchange, parity) forces at each domain edge, and pick `Left`/`Right` $\in \{0, 1, 2, 3\}$ accordingly (Section 5's table). **The physical BC is a statement about the true wavefunction; whether it translates to a Dirichlet or Neumann code for the reduced function $u = R^\alpha \Psi$ depends on your `AlphaFactor` choice** – don't guess, work out the small-R/domain-edge power-law behavior explicitly the way Section 5 does for the 3-body case.

Step 3: Write a plugin

Copy `SechFunctionPlugins.f90` as a template. You need exactly three subroutines conforming to `AdiabaticInterfaces.f90`'s abstract interfaces:

1. A `PotentialProc`: convert the passed angular quadrature points to your physical coordinates, evaluate V and $\partial V/\partial R$ (needed even if you only ever use `CouplingFlag=0` downstream – `CalcHamiltonianGeneric` always computes `dPot` internally, it's just unused when coupling isn't requested).
2. A `GridMakerProcI`: return the angular knot grid at a given R. If your potential has coalescence-type features that sharpen with R, refine around them the way `SechGridMaker` narrows a fixed-width window as $\delta x = r_0/R$.
3. A `RobinCoeffProc`: return the sentinel `1d20` for both outputs unless you genuinely have a Robin (contact-interaction-type) boundary condition, in which case return the real physical log-derivative jump (see `DeltaRobinCoeff` for the worked delta-function example).

Validate the plugin in isolation before trusting it in the full pipeline. This session's most productive debugging technique was writing small, throwaway standalone comparator programs that call the new plugin's `PotentialProc`/`GridMakerProcI` side by side with a trusted reference (either ASB's hardcoded `OneDimChannels`, or the same plugin fed through two different code paths) at a handful of fixed R values, and diffing to machine precision. This caught a dropped-square-root typo in a geometric prefactor that a full end-to-end run would only have manifested as "somewhat wrong" scattering results – much harder to localize.

Step 4: Choose an evaluation strategy

Strategy	When	Caveat
Closed-form analytic (`DeltaFunctionPlugins.f90`-style)	Only if your potential admits one (e.g. a contact/delta interaction, or another exactly solvable model)	Cheapest by far; no ARPACK diagonalization needed at all.
Tabulate + interpolate (`AdiabaticPotential.f90`)	Production runs over many energies/box configurations where the same physics gets reused repeatedly	Cheap per query, but the interpolant's domain is exactly `Rdata(1:NumDataPoints)` from whatever tabulation generated it – **the tabulation's own R-grid must have its first point below the smallest R any box's quadrature will ever query** (Section 8), and a *uniform* tabulation grid will almost certainly under-resolve near-origin behavior for any interesting few-body potential (steep excited-channel curves near $R \to 0$) – use a quadratic or log-spaced tabulation grid.
Direct, on-the-fly diagonalization (`SechAdiabaticPotentialDirect.f90`-style)	Diagnostic runs, cross-checks, or any case where interpolation accuracy is in doubt	No domain-floor issue at all (mirrors `DeltaScat`'s design), but $\sim$0.05 s per quadrature point – **cache per-box results explicitly and never re-evaluate per energy** (Section 7).

Step 5: Box representation

B-spline (`MakeBasis`/`CalcGamLam`) or SVD/DVR (`BuildSVDBox(Generic)`)? SVD boxes need far fewer radial points per box (a handful of DVR nodes vs. a dense B-spline mesh) at the cost of the channel-overlap-matrix bookkeeping and the `GridType` choice (Section 5). If you're using the direct-evaluation strategy and a B-spline box, follow `RMATPROPAdiabaticDirect3Boson.f90`'s pattern exactly (Section 7 below) or the cost will blow up.

Step 6: Grid optimization

- **Angular resolution** (`xNumPoints`/`LegPoints` for the angular diagonalization): match or exceed whatever generated any reference/tabulated dataset you're comparing against. This session found no measurable difference between ϕNumPoints=80 and 200 for a single-point curve check, but always used 200 (matching the original ASB tabulation) for anything meant to be authoritative.
- **Radial box spacing** (`BoxSpacingPower`): linear (1.0, uniform box widths) or quadratic (2.0, clusters boxes near `xStart`). Every driver in this codebase currently hardcodes 1.0 despite several comments suggesting quadratic was considered – worth revisiting for a genuinely fast-varying near-origin potential.
- **Radial tabulation grid** (only relevant to the tabulate+interpolate strategy): as above, quadratic-or-better spacing near $R_{\min}$, not uniform.
- **Box 1's own internal knot grid**: already quadratic-clustered toward `xl` via `GridMaker(...,"quadratic")` in every driver that has one – this is independent of `BoxSpacingPower` (which controls box *boundaries* across the whole chain) and of the tabulation grid (which controls the underlying dataset's resolution). **These are three independent "quadratic" knobs** – don't assume tuning one fixes a resolution problem actually caused by another; this session's "try a quadratic grid" fix for the near-origin interpolation problem specifically meant the *tabulation* grid, and left the other two unchanged.

Step 7: Asymptotic matching / cross sections

`CalcK` assumes your problem's asymptotic channel functions are hyperspherical-Bessel/ Riccati-Bessel type (via `hyperrjry` in `besselnew.f90`), parameterized purely by each channel's threshold energy and effective angular momentum `Leff` (`FitLeff.data`, produced by `NewFitProg.f`'s long-range fit of the tabulated $U_n(R)$ tail to a $L(L+\mathrm{EffDim}-2)/2\mu R^2$-type centrifugal form). **If your problem's long-range tail is not of this form** – e.g. a genuine $1/R$ Coulomb tail between fragments, a dipole-dipole $1/R^3$ long-range interaction, or any other non-centrifugal asymptotic potential – `CalcK`'s matching is *not* directly applicable, and you will need either:

- A different long-range fit routine (analogous to `NewFitProg.f`, but fitting to whatever functional form is actually correct for your tail), feeding `CalcK` an `Leff` that is only an *effective* approximation valid where the true tail has decayed to negligible strength by x_{End} (i.e. push x_{End} far enough that the residual long-range coupling is negligible) – viable if the true tail decays faster than $1/R^2$; or
- A genuinely different final-matching routine using the correct asymptotic reference functions for your tail (Coulomb wave functions instead of Riccati-Bessel, for instance) – this would be

new code, not a parameter change, and is the one place in this whole pipeline where "write a plugin" isn't enough; `CalcK` itself would need a problem-specific sibling.

Once you have a valid K-matrix, $S = (1 - iK)^{-1}(1 + iK)$ (already implemented inline in every driver's energy loop via `CompSqrMatInv`), and cross sections follow from the standard multichannel formulas using `SD%K`/`SD%f`/`SD%sigma` (`DataStructures`' `ScatData` type already has slots for these, though not every driver fills `f`/`sigma` – check the specific driver).

Validate with unitarity before trusting any cross section. Every driver in this pipeline computes $1 - |S_{11}|^2$ (or the general $\|S^\dagger S - I\|_\infty$, in the SVD-generic family) as a running sanity check – it must vanish (up to finite-channel-count/finite-$R_{\max}$ truncation, which should shrink as you add channels or push $x_{\rm End}$ out) for any correctly-implemented closed system. This is the fastest way to catch a wrong asymptotic-matching implementation before trusting any physical number that comes out of it.

6.1 `Leff` must itself be basis-converged, not just the propagation

A cross-check between `RMATPROPAdiabatic.x` (tabulated+interpolated) and `RMATPROPHybridSVDGenericSech3Boson.x` (all-SVD, direct) on the identical equal-mass 3-identical-boson sech^2 physics initially showed the single-open-channel atom-dimer phase shift disagreeing by close to (but not exactly) π across an entire energy scan, plus a residual 12–20° after removing that offset. **Neither turned out to be a code bug.**

- **The $\approx \pi$ offset is expected and harmless.** `delta` is written "unwrapped, no anchor/offset applied" – each driver's phase-continuation starts from its own first-computed K value's principal arctan branch, which is arbitrary. Two independent implementations have no reason to agree to better than $n\pi$; this is not evidence of a bug by itself.
- **The residual traced to `Leff(1)` disagreeing between independently-fit datasets.** `hyperrjry`'s reference-function order is *exactly* `halfd+Leff(mch)-1` (i.e. `Leff(mch)` itself, for `EffDim=2`), so any mismatch in `FitLeff.data`'s fitted `Leff` directly shifts `CalcK`'s asymptotic phase reference by $\approx -\Delta\nu\,\pi/2$ (leading-order Bessel-phase asymptotics) – quantitatively consistent with the observed residual. Two datasets fit over different R windows disagreed (`Leff(1)`$= 0.5106$ over $R \in [40, 50]$ vs. 0.6443 over $R \in [191, 201]$); overriding one to match the other collapsed the π-branch ambiguity to exactly 0 and the residual to $\sim 1°$.
- **`Leff` itself is not converged with too few retained channels.** Even at the *same* $R \in [191, 201]$ fit window, the 5-channel and 20-channel datasets disagree (0.6443 vs. 0.7418) – the long-range tail `NewFitProg.f` fits depends on how complete the angular basis was when the underlying $U_n(R)/Q_{mn}(R)$ curves were generated, not just on the fit window. Rerunning both drivers at 20 retained channels (on a dataset matched on *both* axes, Section 4.4 above) brought the branch offset to exactly 0 and the residual to sub-degree agreement across the whole scan.

Takeaway for validation work: before attributing an SVD-vs-adiabatic (or any two-code-path) disagreement in a derived quantity like a phase shift to a real bug, check whether the two paths are actually consuming *consistent* auxiliary data (`Threshold`/`Leff`), not just running the same nominal physics – and remember that `Leff` converges with channel count independently of how well-converged the propagation/matching itself is.

A related, narrower conditioning pitfall, hit while isolating the above: comparing raw R-

matrices (Rmat $= Z_f^2/b_f$, $b_f = -Z_f'/Z_f$) directly at a single box edge is unreliable near an accidental zero of b_f (a wavefunction turning point) – a ∼1–3% representation difference between two otherwise-correct code paths gets divided by a near-zero b_f and amplified into an $O(1)$ (even opposite-sign) R-matrix discrepancy, even though the wavefunction *shape* agrees well. Diagnosed by shifting the comparison radius by a quarter wavelength $\pi/(2k)$ of the relevant channel's asymptotic oscillation ($k = \sqrt{2\mu E - \text{Threshold}}$) – this moves b_f away from zero and the discrepancy shrinks in proportion, confirming a conditioning artifact rather than a bug. Prefer comparing the branch-continued phase shift ($\arctan K$, unwrapped) over raw K or raw Rmat for exactly this reason – both have poles/zeros that a phase-shift comparison sidesteps.

7 Cost Discipline for Direct (No-Interpolation) Potential Evaluation

Every direct-evaluation call is a full ARPACK shift-invert diagonalization, ∼0.05 s at 200-point angular resolution (measured this session, SechCurvesSVD.x: 400 points in ∼20 s wall time). This is fine for a one-time box construction ($N_{\text{boxes}} \times$ (xNumPoints $- 1$) $\times$ LegPoints points total – e.g. ∼9000 points, ∼7–8 minutes, for a 100-box/10-knot/10-Legendre-point B-spline chain) but **catastrophic if repeated per energy**. RMATPROPAdiabatic.f90's tabulate+interpolate last-box handling rebuilds the potential fresh every energy (harmless there, since an interpolation lookup is cheap) – **do not copy that pattern literally into a direct-evaluation driver.** RMATPROPAdiabaticDirect3Boson.f90's fix: give the last box its own dedicated BPD (BPDLast), call SetAdiabaticPotentialDirect on it exactly once outside the energy loop, and inside the loop only call CalcGamLam again (cheap dense linear algebra on the already-computed Pot/Pcoup arrays, no re-diagonalization) to pick up that energy's NumOpenR.

Two implementation bugs this session hit building this, worth remembering if you write another direct-evaluation provider:

- **Radial vs. angular LegPoints are different quantities.** The box's own radial Gauss-Legendre quadrature (how many R points per box sub-interval – must match the driver's global LegPoints/xLeg from the Quadrature module, already set up via GetGaussFactors) is unrelated to OneDimChannelsGeneric's own LegPoints argument (angular integration resolution inside each diagonalization). Reusing one variable for both is an easy, silent mistake.
- **Close a scratch file before reopening it for reading.** If you write to a unit inside a called subroutine (e.g. OneDimChannelsGeneric writing to Pfile/Qfile) and then OPEN that same unit number again in the caller without an intervening CLOSE, the reopen does not rewind the file position – subsequent reads start from wherever writing left off (i.e. immediately hit end-of-file), even though the file's on-disk content is completely correct. The error message ("End of file" at the *read* statement) is easy to misdiagnose as a data-generation bug rather than a file-handling one.

Also be aware that ARPACK can converge fewer than the requested number of eigenvalues at a difficult R (nQ=MIN(NumStates,iparam(5)) $<$ NumStates in OneDimChannelsGeneric); a consumer that assumes a fixed NumStates$\times$ NumStates block per record in the P/Q output files will break unless the writer pads short blocks with zero – AdiabaticSolverGeneric.f90 was patched this session to always zero-pad and write the full block, since this was the first code path in the workspace to ever round-trip that data through a fixed-width file format.

8 Known Gotchas – Quick Reference

1. **Literal $R = 0$ is unusable for direct potential evaluation.** The hyperspherical kinetic prefactor $m = -1/(2\mu R^2)$ is genuinely singular there (confirmed: crashes LAPACK's `DLASCL` with an illegal-parameter error, and ARPACK's `_saupd` with `info=-9999`, the first time this was tried this session), and that single failure then poisons *every subsequent* R in the same run because `OneDimChannels/ OneDimChannelsGeneric` reuse residual/state arrays across the R-sweep without resetting. Use a tiny nonzero `RMin` (e.g. `1d-6`) for any driver that tabulates/evaluates a potential directly at explicit R points – this is a genuinely different situation from the DVR mechanisms in Section 5, which structurally never evaluate exactly at a_1 regardless of its value.

2. **A dataset's own tabulation resolution near $R = 0$ can silently be far too coarse.** A uniform 400-point grid from $R_{\min} = 10^{-6}$ to $R_{\max} = 50$ gives $R_{\text{data}}(2) \approx 0.125$ – a five-order-of-magnitude gap in R (and up to ten orders of magnitude in $U_n(R)$ for steep excited channels) with *nothing* tabulated in between. Any interpolated value queried inside that gap is meaningless. Use a quadratic (or finer) tabulation grid; even then, verify by direct comparison against a no-interpolation reference before trusting results that depend on near-origin resolution.

3. **The interpolant's own domain floor is a hard constraint, not just an accuracy concern.** `AdiabaticPotential.f90` builds `UInterp/PInterp/QInterp` over `Rdata(1:NumDataPoints)` (this session changed this from `Rdata(2:NumDataPoints)`, which dropped the first tabulated point unconditionally for a reason – Runge-ringing near a genuine Coulomb-type near-origin divergence – that turned out to be dataset-specific and, worse, already independently guarded by `RcutoffAnalytic` for the *only* channels it actually applies to). **Rule: `Rdata(1)` must lie in $0 < R < R_{\text{quad}}(1)$, where $R_{\text{quad}}(1)$ is the smallest R any box's own quadrature will ever query** – work this out from your box-1 width and internal quadratic-grid clustering (Section 6, Step 6) before generating a dataset, not after hitting a `dbsval` domain-edge crash.

4. **Don't infer "genuinely Coulomb-divergent" from `Threshold` < 0 alone.** `RMATPROPAdiabatic.f90`'s `CoulombC/gamma0` near-origin substitution is an equal-mass delta-function-specific formula; a short-range-potential bound-pair channel (e.g. the sech^2 well's dimer channel) also has negative threshold without needing (or tolerating) that substitution. Gated behind an explicit `ApplyCoulombC` input flag (default off) rather than inferred from the sign of `Threshold` – keep it that way for any new dataset.

5. **`GridRebuildEveryR` must be `.FALSE.` for SVD/DVR box construction.** `GenericSVDChannelBasis.f90`'s own header explains why: the channel-overlap-matrix trick assumes every DVR node in a box shares one angular basis, anchored at the box's own rightmost R. Passing `.TRUE.` silently corrupts the cross-node overlap without an error message – the symptom is plausible-looking but wrong physics (huge, inconsistent numbers), not a crash. `.TRUE.` is correct for a plain curve dump (evaluating the potential independently at each of many explicit R values, no box-sharing assumption involved) but wrong for building an actual box.

6. **`GridMakerHHL`'s 4-segment refinement scheme assumes the interior coalescence angle $\phi_{23} = \pi/6$ is *strictly interior* to the domain.** If a symmetry-reduced domain's right edge coincides with ϕ_{23} exactly, the standard segment-3 degenerate-width guard produces a *reversed* (negative-width) segment instead of gracefully shrinking to zero, corrupting the B-spline knot vector (non-monotonic) and crashing ARPACK/LAPACK downstream with no direct clue pointing at the grid maker. Fixed this session in both repos' copies of `adiabaticSolver1D.f90` with an explicit `xMax .le. phi23` special case (falls back to a plain 2-segment coarse+refined-

near-edge split); confirmed a no-op for the existing validated wide-domain case.

7. **The $\tilde{Q}$ sign/normalization convention.** This codebase's `Qmat.dat/BPD%Pot` convention is `Pot`$(m,n) = +\tilde{Q}_{mn}/(2\mu) + \delta_{mn}U_m$ (verified directly: $2\mu U + \tilde{Q} - 0.25/R^2$ is flat to 10^{-12} out to $R = 200$ for the validated delta-function dataset, while the opposite sign visibly drifts). `OneDimChannelsGeneric`'s own $Q = -P \cdot P$ (via `dgemm`) is exactly this $\tilde{Q}$ (not the "full" $Q = P^2 + \partial_R P$ from the hyperspherical-Laplacian derivation) – don't "fix" this to match a different sign convention found elsewhere in the workspace (e.g. `CABS.f90/VQmat.dat` uses an internally opposite convention from this repo's own).

8. `Left/Right` **BC codes 0–3 are shared across at least three different physical meanings depending on which axis you're describing**: the angular BC at a hyperangle domain edge (Section 6), the radial BC at a box's own inner/outer edge (Section 5), and (for the SVD family) which `GridType` a box uses. Keep these straight when reading or writing a driver – the same integer `2` means "open, both boundary functions retained" in every context, but *which* boundary and *which* physical condition it ends up encoding depends entirely on which of these three axes you're on.

9. `FitLeff.data` **has two live formats and a naive fixed-column read silently misreads** `Leff`**.** The legacy format is 5 columns (`j,Threshold,residual,Leff,FitRangeMin`); current `NewFitProg.f` output drops the always-zero `residual` placeholder, giving 4 columns (`j,Threshold,Leff,FitRangeMin`). A plain list-directed `READ` written for one format against a file in the other silently shifts every field left by one – no read error, but `Leff` ends up holding `FitRangeMin` (or the residual placeholder) instead, quietly corrupting `CalcK`'s asymptotic matching (confirmed to have happened, more than once, with the `3bodydata_sech2_3boson` dataset). `AdiabaticPotential.f90`'s `ReadAdiabaticData` and `RMATPROPHybridSVDGenericSech3Boson.f90` both guard against this by reading each line into a string buffer and trying the 5-column parse first (an internal `READ`, so a short 4-column line reliably fails with nonzero `IOSTAT` instead of spilling into the next record), falling back to the 4-column parse on failure. Copy this pattern in any new code that reads `FitLeff.data` directly.

9 Validation Playbook

No automated test suite exists for this codebase; validation is against known physics invariants and literature benchmarks. Techniques that proved effective this session, roughly in order of how often to reach for them:

1. **Unitarity first.** $1 - |S_{11}|^2 \to 0$ (or the general $\|S^\dagger S - I\|$) is free (already wired into every driver) and catches a wide class of bugs – wrong asymptotic matching, a sign error in the propagation, a genuinely singular linear-algebra step – before you even need a literature comparison.

2. **Before/after regression via** `git checkout`**.** When changing a shared/core file, capture a validated benchmark's output, `git checkout --` the file back to its last-committed state, rerun, diff. Bit-for-bit (or machine-precision) identical output on an *already-validated* benchmark is strong evidence a change is a true no-op for existing behavior, before trusting it on new physics.

3. **Standalone A/B comparators.** A small throwaway program calling two code paths (e.g. a new generic plugin vs. the hardcoded reference it's meant to replace) at the same handful of R

values, diffing to machine precision, isolates a bug to a specific routine far faster than running the whole pipeline and eyeballing whether the final answer "looks about right." This is how the dropped-square-root `RPair` typo was found this session – it was invisible in end-to-end scattering output (just "somewhat wrong"), obvious once isolated to the raw potential values.

4. **Literature/exact-formula comparison** where available (Mehta-Shepard atom-dimer K-matrix, Amaya-Tapia/Larsen/Popiel coupled-channel phase shifts, McGuire's exact elastic-unitarity result) – the gold standard, but only available for the handful of exactly solvable benchmark systems (equal-mass delta-function, free particle).

5. **Independent numerical cross-check between fundamentally different code paths** for the *same* physics – tabulate+interpolate vs. direct diagonalization (Section 7), or B-spline-box vs. SVD-box – when no closed-form answer exists. Genuinely independent only if the two paths don't share the actual evaluation code (two callers of the same `OneDimChannelsGeneric` engine, differing only in potential/grid, are *not* independent of each other in the way "interpolated vs. exact" or "`DeltaScat` vs. `R-matrix-prop`'s own engine" are).

6. **Before trusting a two-code-path disagreement, check the auxiliary data matches, not just the nominal physics.** Two drivers reading independently-generated datasets for "the same" system can disagree in `Threshold`/`Leff` even when both are individually valid (Section 6.1) – confirm this isn't the whole story (e.g. by temporarily forcing one driver to use the other's values) before spending time hunting for a code bug that isn't there.

7. **Compare the branch-continued phase shift (**$\arctan K$**, unwrapped), not raw** K **or a raw** R**-matrix, when validating against another code path.** Both $K = \tan\delta$ and $\mathrm{Rmat} = Z_f^2/b_f$ have a genuine pole/zero structure in δ/b_f that a small, otherwise-unremarkable representation difference between two correct implementations can land squarely on, producing an $O(1)$ or even opposite-sign discrepancy that looks like a bug but isn't (Section 6.1). If a raw-K/raw-Rmat discrepancy shrinks when the comparison radius is shifted by a fraction of the relevant channel's asymptotic wavelength, it was a conditioning artifact, not a bug.